

Estrutura de Dados I

Prof.: Michele Nasu Tomiyama Bucci

Tipo Abstrato de Dados

- Um **Tipo Abstrato de Dados** é uma especificação de um conjunto de dados e operações que podem ser executadas sobre esses dados.
- Além disso, é uma metodologia de programação que tem como proposta reduzir a informação necessária para a criação/programação de um algoritmo através da abstração das variáveis envolvidas em uma única entidade fechada.
- Com operações próprias à sua natureza.

- A idéia de **tipo abstrato de dados** é desvincular o tipo de dado (valores e operações) de sua implementação.
- Um **tipo abstrato de dados** pode ser definido matematicamente pelo par (V, O) onde:
 - V é um conjunto de valores
 - O um conjunto de operações sobre estes valores.
- Ex.: tipo real
- $V = \mathbb{R}$
- $O = \{+, -, *, /, =, <, >, \leq, \geq\}$

Estruturas Estáticas e Dinâmicas

- Quando trabalhamos com estruturas de dados, uma das questões importantes é **como e quando a memória necessária para armazenar os dados é reservada**.
- De maneira geral, podemos trabalhar com **armazenamento estático** ou **armazenamento dinâmico**.
- Essa diferença é importante porque influencia a flexibilidade, o uso da memória e a forma como as estruturas de dados são implementadas.

- No **armazenamento estático**, o espaço de memória necessário para armazenar os dados é definido previamente. Em outras palavras, o tamanho da estrutura é determinado antes ou no momento de sua criação e permanece fixo durante sua utilização.
- Um exemplo clássico é um **vetor (array) de tamanho fixo**.
- No Java, a sintaxe para declara um vetor é:

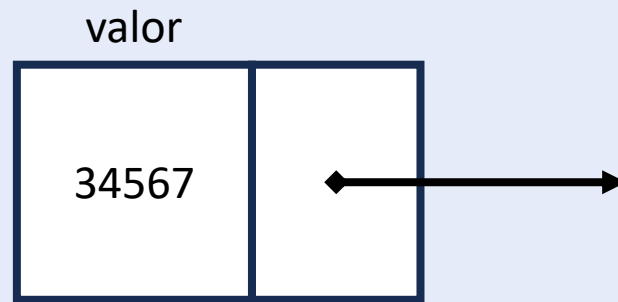
```
tipoDados[] nomeVetor = new tipoDados[tamanho];
```

- Exemplo:

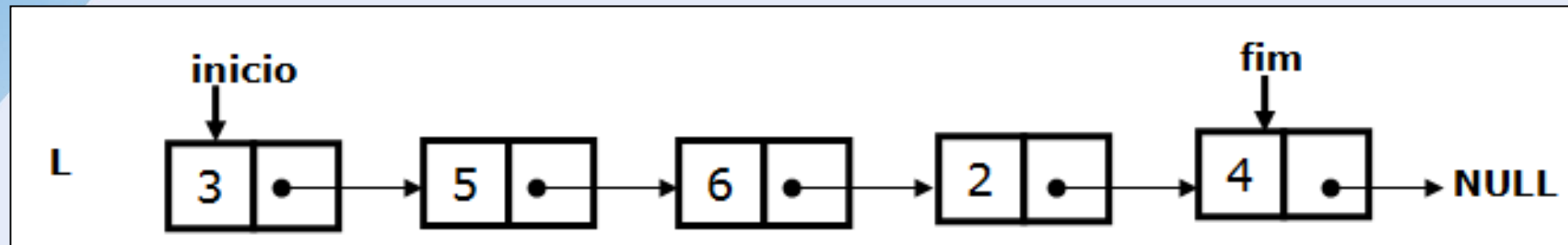
```
int[] vet = new int[10];
```

- Uma característica importante do armazenamento estático é a **previsibilidade**.
- Sabemos previamente quanto espaço será utilizado. Isso pode facilitar o gerenciamento da estrutura e permitir acesso direto aos elementos.
- Por outro lado, existe uma limitação: se a quantidade de dados aumentar além da capacidade inicialmente definida, será necessário criar uma estrutura maior ou utilizar outro mecanismo para armazenar os novos elementos.

- No **armazenamento dinâmico**, a memória é alocada e liberada durante a execução do programa, conforme a necessidade.
- Isso permite que uma estrutura de dados **cresça ou diminua de acordo com a quantidade de elementos armazenados**.
- Um exemplo clássico é uma **lista encadeada**.
- Uma lista encadeada é formada por **nós**. Cada nó normalmente contém:
 - o dado armazenado;
 - uma referência (ou ponteiro) para outro nó.



- Assim, cada novo elemento pode ser armazenado em uma nova posição de memória e conectado à estrutura.
- Dessa maneira, formando estruturas encadeadas, crescendo conforme a necessidade.



- A escolha da forma de armazenamento está relacionada ao problema que queremos resolver.
 - Se sabemos antecipadamente a quantidade de elementos e precisamos de acesso rápido por índice, uma estrutura baseada em armazenamento contíguo pode ser uma boa opção.
 - Por outro lado, quando a quantidade de elementos varia muito durante a execução, estruturas dinâmicas, como listas encadeadas, podem oferecer maior flexibilidade.

TAD NÓ

- A estrutura nó é composta por 2 campos: **valor** e **prox**.
- O campo **valor** armazena a informação.
- O campo **prox** armazena a localização do próximo nó do encadeamento.

```
1 public class No {  
2     private Object valor;  
3     private No prox;  
4 }
```

A classe No será utilizada em todas as estruturas vistas nesta disciplina.

- Métodos construtores:

- Serão utilizados sempre que uma nova informação será armazenada na estrutura. Então é necessário criar um novo **No** para guardar esse novo valor e conectar com a estrutura.

```
5      public No(Object valor, No prox) {
6          this.valor = valor;
7          this.prox = prox;
8      }
9      public No(){
10
11      }
```

- Métodos de acesso **gets** e **sets**:

```
13     public Object getValor() {  
14         return valor;  
15     }  
16  
17     public void setValor(Object valor) {  
18         this.valor = valor;  
19     }  
20  
21     public No getProx() {  
22         return prox;  
23     }  
24  
25     public void setProx(No prox) {  
26         this.prox = prox;  
27     }  
28 }
```

